Building a Large-Scale Machine Learning Pipeline for Job Recommendations

Posted on April 26, 2016 by Preetha Appan

(Editor's note: This post was originally published on oreilly.com (https://www.oreilly.com/ideas/algorithms-and-architecture-for-job-recommendations).)

With 200 million unique visitors every month, Indeed (http://www.indeed.com/) relies on a recommendation engine that processes billions of input signals every day — resulting in more external online hires (http://blog.indeed.com/2015/04/29/indeed-helps-more-people-get-hired-than-any-other-job-site/) than any other source.

To create this recommendation engine, we started with a minimum viable product (https://en.wikipedia.org/wiki/Minimum_viable_product) (MVP) built with Apache Mahout (http://mahout.apache.org/) and evolved to a hybrid, offline + online pipeline. Along the way, our changes affected product metrics, and we addressed challenges with incremental modifications to algorithms, system architecture, and model format. The lessons we learned in designing this system can apply to any high-traffic machine learning application.

From search engine to recommendation

Indeed's production applications run in many data centers around the world. Clickstream data — and other application events from every data center — are replicated into a central HDFS repository (https://engineering.indeedblog.com/blog/2012/11/logrepo-enabling-data-driven-decisions/), based in our Austin data center. We compute analytics and build our machine learning models from this repository.

Our job search engine is simple and intuitive, with two inputs: keywords and location. The search results page (http://www.indeed.com/q-software-engineer-l-Austin-jobs.html) displays a list of matching jobs, ranked by relevance. The search engine is the primary way our users find jobs on Indeed.

Our decision to go beyond search and add job recommendations — as a new mode of interaction — was based on several reasons:

- 25% of all searches on Indeed specify only a location and no search keywords. Many job seekers aren't sure what keywords to use in their search.
- When we deliver targeted recommendations, the job seeker's experience is personalized.
- Recommendations can help even the most sophisticated user discover additional jobs that their searches would not have uncovered.
- With recommendations driving 35% of Amazon sales and 75% of Netflix content views (http://www.mckinsey.com/industries/retail/our-insights/how-retailers-can-keep-up-with-consumers), it's clear they provide added value.

Recommending jobs is significantly different than recommending products or movies. Here are just a few of the things we took into careful consideration as we built our engine:

Rapid Inventory Growth. We aggregate millions of new jobs on Indeed every day. The set of recommendable jobs is changing constantly.

New Users. Millions of new job seekers visit Indeed every day and begin their job search. We want to be able to generate recommendations with very limited user data.

Churn. The average lifespan of a job on Indeed is around 30 days. Content freshness matters a lot, because the older the job, the more likely it is to have been filled.

Limited Supply. One job posting is usually meant to hire one individual. This is different from books or movies, where as long as there is inventory it can be recommended to many users at the same time. If we over-recommend a job, we could bombard an employer with thousands of applications.

How to approach recommendation algorithms

Recommendations are a matching problem. Given a set of users and a set of items, we want to match users to their preferred items. There are two high-level approaches to this: content-based and behavior-based. They each have pros and cons, and there are also ways to combine these approaches to take advantage of both techniques.

Content-based approaches use data, such as user preferences and features of the items being recommended, to determine the best matches. For recommending jobs, using keywords of the job description to match keywords in a user's resume is one content-based approach (note that users can upload their resume to the Indeed site). Using keywords in a job to find other similar jobs is another way to implement content-based recommendations.

Behavior-based approaches leverage user behavior to generate recommendations. These approaches are domain-agnostic, meaning the same algorithms that work on music or movies can be applied to the jobs domain. Behavior-based approaches do suffer from a cold start (https://en.wikipedia.org/wiki/Cold_start) problem. If you have little user activity, it is much harder to generate good quality recommendations.

Mahout collaborative filtering

We started by building a behavior-based recommendation engine because we wanted to leverage our existing job seeker traffic and click activity. Collaborative filtering algorithms (https://en.wikipedia.org/wiki/Collaborative_filtering) are well understood in this space.

Our first attempt at personalized recommendations was based on Apache Mahout's user-to-user collaborative filtering (https://mahout.apache.org/users/recommender/userbased-5-minutes.html) implementation. We fed clickstream data into a Mahout builder that ran in our Hadoop cluster, and produced a map of users to

recommended jobs. We built a new service to provide access to this model at runtime, and multiple client applications accessed this service to recommend jobs.

MVP results and roadblocks

As an MVP, the behavior-based recommendation engine showed us that it is important to start small and iterate. Building this system quickly and getting it in front of users demonstrated that these recommendations were useful to job seekers. However, we ran into several immediate problems using Mahout on our traffic:

- The builder took around 18 hours on Indeed's 2013 clickstream, which is about 3X smaller than present day.
- We could only run the builder once a day, which meant that millions of new users joining Indeed every day wouldn't see recommendations until the next day.
- Millions of new jobs aggregated on Indeed were not visible as recommendations until the builder ran again.
- The model we produced was a large map, which was around 50GB, and took several hours to copy over a WAN from the data center where it was built to our data centers around the globe.
- Mahout's implementation exposed a few tweakable parameters, like similarity thresholds. We could tweak the parameters of the algorithm, but we wanted the flexibility to test entirely different algorithms.

Implementing MinHash for recommendations

We addressed the most important problem first: the builder was too slow. We found that user-to-user similarity in Mahout is implemented by comparing every user to every other user in n^2 time. For only U.S. traffic in Indeed (50 million UVs), this amounts to $15 * 10^{15}$ comparisons, which is intractable. This calculation is also batch in nature. Adding a new user or a new click event requires recalculating all similarities.

We realized that recommendations were an inexact problem. We were looking for ways to find the closest users to a given user, but we didn't need 100% accuracy. We looked for ways to approximate similarity without having to calculate it exactly.

Principal contributor Dave Griffith came across MinHash from a Google News academic paper (http://www2007.org/papers/paper570.pdf). MinHash, or minwise independent permutations, allows approximating Jaccard similarity (https://en.wikipedia.org/wiki/Jaccard_index). Applying this measure to jobs that two users clicked on at Indeed, we see that the more jobs these two users have in common, the higher their Jaccard similarity. Calculating Jaccard similarity for all pairs of users is $O(n^2)$, and with MinHash, we can reduce this down to $O(n)$.

The MinHash of a set of items, given a hash function *h1*, is defined as taking the hash of all items in that set using that hash function, and then taking the minimum of those values. A single hash function is not sufficient to approximate the Jaccard similarity because the variance is too high (https://en.wikipedia.org/wiki/MinHash).

We have to use a family of hash functions to reasonably approximate Jaccard distance. With a family of hash functions, MinHash can be used to implement personalized recommendations with tweakable Jaccard similarity thresholds.

Mining Massive Datasets (https://www.coursera.org/course/mmds), a recent Coursera course from Stanford professors Leskovec, Rajaraman, and Ullman, explains MinHash in great detail. Chapter 3 (http://infolab.stanford.edu/~ullman/mmds/ch3.pdf) (PDF) of their book, “Mining Massive Datasets,” explains the mathematical proof behind MinHash.

Our implementation of MinHash for recommendations involved the following three phases:

Phase 1: Signature calculation/cluster assignment

Every job seeker is mapped to a set of *h* clusters, corresponding to a family of hash functions H. The following pseudocode shows this:

```
H = {H1, H2, ..H20}
for user in Users
     for hash in H
          minhash[hash] = min{x∈Items_i| hash(x)}
```

Where `H` is a family of *h* hash functions. At the end of this step, each user is represented by a signature set: a cluster consisting of *h* values.

Phase 2: Cluster expansion

Users that share the same signature are considered similar, and their jobs are cross recommended to each other. We expand each cluster with all the jobs from each user in that cluster.

Phase 3: Recommendation generation

To generate recommendations for a given user, we union all jobs from the *h* clusters that the user is in. We remove any jobs that this user has already visited to get the final set of recommended jobs.

Recommending jobs to new users

MinHash’s mathematical properties allow us to address recommending jobs to new users and recommending new jobs to all users. We update the MinHash signature for users incrementally as new clicks come in. We also maintain a map in memory of new jobs and their MinHash clusters. By keeping these two pieces of data in memory, we are able to recommend jobs to new users after they click on a few jobs. As soon as any new jobs posted throughout the day receive clicks, they are recommended to users.

After transitioning to MinHash, we had a hybrid recommendation model — consisting of an offline component that builds daily in Hadoop — and an online component implemented in memcache — consisting of the current day's click activity. Both models are combined to compute the final set of recommendations per user. The recommendations for each user became more dynamic after we made this change, because they would update as users clicked on jobs that interested them.

With these changes, we learned that we could trade off a little bit of accuracy for a lot of performance. We also learned to complement a slower offline model with an online model for fresher results.

Engineering infrastructure improvements

The recommendation model that contained a map from each user to their recommendations was a large monolithic file. Because jobs are local to each country, we first attempted to shard our data into zones based on approximate geographic boundaries. Instead of running one builder for the entire world, we ran one builder per zone. Each zone consisted of multiple countries. As an example, the East Asian zone contained recommendations for China, Japan, Korea, Hong Kong, Taiwan, and India.

Even after sharding, some of our zones produced data files that were too big and took hours to copy from our Austin Hadoop cluster over a WAN to a remote data center in Europe. To address this, we decided to ship recommendation data incrementally rather than once per day. We reused sequential write ahead logs (https://engineering.indeedblog.com/blog/2013/10/serving-over-1-billion-documents-per-day-with-docstore-v2/) and log structured merge trees to implement this. This was already validated in other large production applications at Indeed, like our document service.

Instead of producing a large model file, we modified our builder to write small segments of recommendation data. Each segment file is written using sequential I/O and optimized for fast replication. These segments get reassembled into a log structured merge tree (https://en.wikipedia.org/wiki/Log-structured_merge-tree) in recommendation services running in remote data centers.

This infrastructure change caused users to see their new recommendations hours faster in remote data centers. From our A/B testing of this change, we saw a 30% increase in clicks due to the fact that users received newer recommendations faster.

This improvement demonstrated that engineering infrastructure improvements can make as much of an impact on metrics as algorithm improvements.

A/B testing velocity

Building out the pipeline to compute and update recommendations was only the beginning. To improve the coverage and quality of recommendations, we needed to increase our A/B testing velocity.

We were making many decisions in the builder to tune the final set of recommendations. These decisions

included similarity thresholds, the number of jobs to include in an individual's recommendations, and different ways to filter out poor quality recommendations. We wanted to tweak and optimize every aspect of computing recommendations, but to do so would require building and shipping a new model per algorithm tweak. Testing multiple improvement ideas meant many times more disk and memory usage on the servers that handled requests from users.

We began to improve our A/B testing velocity by shipping the individual components of the recommendation calculation, rather than the final results. We changed the recommendation service to perform the final calculation by combining the pieces, instead of simply reading the model and returning results. Critical subcomponents of recommendations are cluster assignments per user, the mapping from each cluster to jobs in that cluster, and a blacklist for each user that contained jobs that should not be recommended for them. We modified our builder to produce these components and modified our service to put them together at request time to return the final list of recommendations.

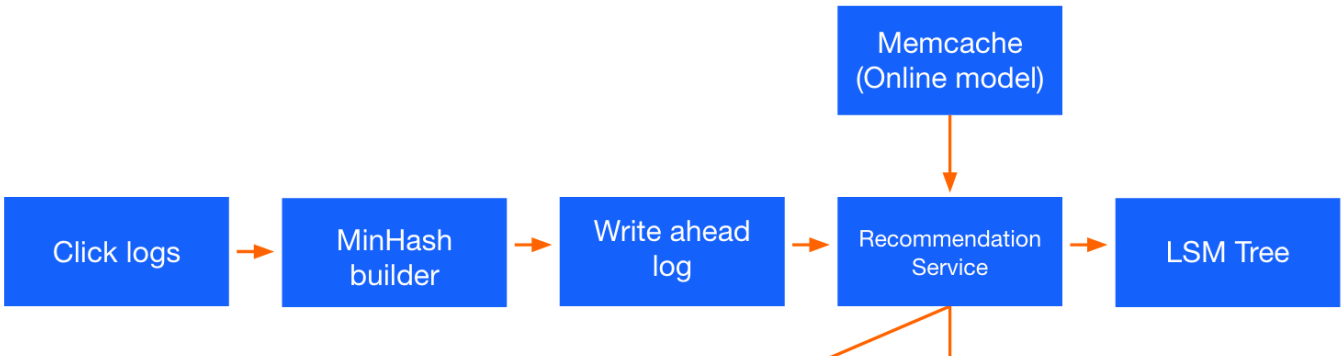
By implementing this architectural change, we only shipped subcomponents that changed per A/B test. For example, if the test only tweaked what jobs got removed from a user's recommendations, we would only ship the blacklist for the test group.

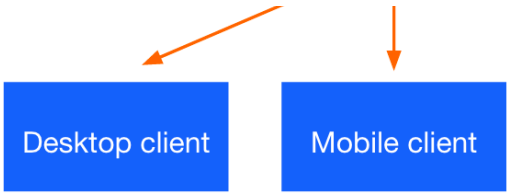
This change improved A/B testing velocity by orders of magnitude. We were able to test and validate several ideas that improved the quality and coverage of recommendations in a short period of time. Previously, we averaged testing one improvement in the model every quarter because of the overhead in setting up our experiments.

Our experience shows that A/B testing velocity should be considered when designing large machine learning systems**.** The faster you can get your ideas in front of users, the faster you can iterate on them.

This post summarizes a series of algorithmic and architectural changes we made as we built our recommendation engine. We build software iteratively at Indeed — we start with an MVP, learn from it, and improve it over time. As a result of these changes, job recommendations grew from a small MVP to contributing 14% of all clicks on Indeed, which is up from 3% in early 2014.

Architecture





(http://engblogs.wpenginepowered.com/wp-content/uploads/2016/04/recommendations-architecture.png)

Architecture diagram for Indeed's recommendation engine

Conclusion

Moving forward, we continue to refine our recommendation engine. We are prototyping a model using Apache Spark (http://spark.apache.org/). We are building an ensemble of models, and we are refining our optimization criteria to combat popularity bias (http://knowledge.wharton.upenn.edu/article/recommended-for-you-how-well-does-personalized-marketing-work/).

« A Funny Thing Happened on the Way to Java 8 (https://engineering.indeedblog.com/blog/2016/09/job-search-web-app-java-8-migration/)

A Bounty of Security » (https://engineering.indeedblog.com/blog/2016/03/a-bounty-of-security/)

@IndeedEng Talks » (/talks)

Indeed Open Source » (https://opensource.indeedeng.io/)

Indeed University » (https://engineering.indeedblog.com/indeed-university/)

Search

Categories

A/B Testing (https://engineering.indeedblog.com/blog/category/a-b-testing/) 6
Agentic IAM (https://engineering.indeedblog.com/blog/category/security/agentic-iam/) 1
Analytics (https://engineering.indeedblog.com/blog/category/analytics/) 6

Big Data (https://engineering.indeedblog.com/blog/category/big-data/) 7
Coaching (https://engineering.indeedblog.com/blog/category/coaching/) 3
Conference Participation (https://engineering.indeedblog.com/blog/category/conference-participation/) 3
CPU Throttling (https://engineering.indeedblog.com/blog/category/cpu-throttling/) 2
Data Science (https://engineering.indeedblog.com/blog/category/data-science/) 14
Engineering (https://engineering.indeedblog.com/blog/category/engineering/) 34
Engineering Culture (https://engineering.indeedblog.com/blog/category/engineering-culture/) 29
Machine Learning (https://engineering.indeedblog.com/blog/category/machine-learning/) 4
Open Source (https://engineering.indeedblog.com/blog/category/open-source/) 35

Toggle More

Archives

March 2026 (https://engineering.indeedblog.com/blog/2026/03/) (2)
November 2025 (https://engineering.indeedblog.com/blog/2025/11/) (1)
June 2025 (https://engineering.indeedblog.com/blog/2025/06/) (1)
August 2024 (https://engineering.indeedblog.com/blog/2024/08/) (1)
July 2024 (https://engineering.indeedblog.com/blog/2024/07/) (1)
January 2024 (https://engineering.indeedblog.com/blog/2024/01/) (1)
April 2022 (https://engineering.indeedblog.com/blog/2022/04/) (2)
January 2022 (https://engineering.indeedblog.com/blog/2022/01/) (1)
June 2021 (https://engineering.indeedblog.com/blog/2021/06/) (1)
March 2021 (https://engineering.indeedblog.com/blog/2021/03/) (1)
February 2021 (https://engineering.indeedblog.com/blog/2021/02/) (1)
December 2020 (https://engineering.indeedblog.com/blog/2020/12/) (1)

Toggle More

Back to top

About Indeed (http://www.indeed.com/about) -

Find Jobs (http://www.indeed.com/?utm_source=publisher&utm_medium=organic_listings&utm_campaign=affiliate) -

Contact Indeed (http://www.indeed.com/support?utm_source=publisher&utm_medium=organic_listings&utm_campaign=affiliate)

©2026 Indeed (http://www.indeed.com/) - Cookies, Privacy and Terms of Service (http://www.indeed.com/legal)

Do Not Sell My Personal Information (https://www.indeed.com/legal/ccpa-dns) - Accessibility at Indeed (https://www.indeed.com/accessibility?hl=en)